

FTSP+: A MAC Timestamp independent flooding time synchronization protocol

Heder Dorneles Soares¹, Raphael Pereira de Oliveira Guerra¹,
Célio Vinicius Neves de Albuquerque¹

¹Instituto de Computação – Universidade Federal Fluminense (UFF)
Av. Gal. Milton Tavares de Souza, São Domingos – Niterói – RJ – Brasil

{hdorneles, rguerra, celio}@ic.uff.br

Abstract. *Wireless sensors networks are distributed system composed of battery-powered nodes with low computational resources. Like in many distributed systems, some applications of WSN require time synchronization among their nodes. In the particular case of WSN, synchronization algorithms must respect the nodes computational constraints. The well known FTSP protocol is famous for achieving nanosecond precision with low overhead. However, it relies on MAC timestamp, a feature not available in all hardware. In this work, we propose MAC timestamp independent version in order to extend and adapt FTSP to work on hardware that do not have MAC timestamp while keeping the low overhead and high synchronization precision. Our results we estimate an average synchronization error of $1.508\mu\text{s}$ per hop, while adding a correction message.*

1. Introduction

Wireless Sensor Networks (WSNs) are composed of small computational devices equipped with an antenna for wireless communication, one or more kind of sensors, and a small CPU for simple data processing [Baronti et al. 2007]. These devices are usually called *motes*. Due to the limited radio signal range and energetic constraint, WSNs have restrictions and unique characteristics that differ from traditional networks and distributed systems [Arampatzis et al. 2005].

WSNs have several applications in many diverse fields. Sensors deployment in military applications have always been widespread, so the introduction of motes was a natural incorporation to the advancement of systems already used. Applications enhanced with WSN include tracking enemy and targets [Yang and Sikdar 2003], monitoring of vehicles [Sinopoli et al. 2003], countersniper system [Simon et al. 2004], and surveillance systems [Gui and Mohapatra 2004]. Environmental monitoring also provide opportunities to apply WSN. Our environment has a lot of information that play an important role in our quality of life, such as quality of air, water, sound and solar radiation to which we are exposed directly and affect our health [Oliveira and Rodrigues 2011, Cardell-Oliver et al. 2004]. Increasing interest in green computing has led to concerns with energy consumption of IT facilities [Chong et al. 2014], and WSNs play a strategic role in monitoring and controlling these environments [Zanatta et al.].

Some of those applications require time synchronization mechanisms with good accuracy and scalability, all this complying with their low computational resources and

energy availability [Zanatta et al. , Simon et al. 2004, Yang and Sikdar 2003]. Flooding Time Synchronization Protocol (FTSP) is the most popular time synchronization algorithm for WSN [Maróti et al. 2004]. It is fault-tolerant, achieves high accuracy ($\sim 1, 5\mu s$ per hop) utilizing timestamps in low layers of the radio stack, and saves energy using a linear regression technique to compensate clock skews with few exchanges of synchronization messages. However, MAC layer timestamping is not a standardized feature, and hence, not interoperable among different hardware and physical layer protocols. There is an effort from Google to standardize MAC layer timestamping in WSN [Wang and Ahn 2009], but so far there is not much compliance.

Many synchronization protocols use MAC timestamp: some have less accuracy than FTSP, focus at other problems and make more restrictive assumptions [Ganerwal et al. 2003, van Greunen and Rabaey 2003]; others can achieve better accuracy between distant nodes [Nazemi Gelyan et al. 2007, Lenzen et al. 2009, Sommer and Wattenhofer 2009]. Elson et al. proposed the Reference Broadcast Synchronization [Elson et al. 2002] (RBS) to eliminate uncertainty of the sender without MAC timestamp by removing the sender from the critical path. The idea is that a third party will broadcast a beacon to all receivers. The beacon does not contain any timing information; instead the receivers will compare their clocks to one another to calculate their relative phase offsets. It has $\sim 30\mu s$ error per hop and is independent of MAC timestamp. Ranganathan and Nygard offer a good overview of these protocols [Ranganathan and Nygard 2010].

In this paper, we propose a modified version of FTSP, called FTSP+, in order to work without MAC timestamp. For that, we use application level time-stamping on synchronization messages, which leads to a well-known problem: the time elapsed between time-stamping the packet and gaining the wireless medium reduces synchronization accuracy. We circumvent this problem using medium access interrupt handlers on the sender to measure the time needed to gain the medium, and correction messages to reduce sender side time uncertainty.

Our experimental results show that we only double FTSP synchronization inaccuracy, a great result compared to RBS which is 10 times worse. We also present a quantitative evaluation of medium access and processing delays on TinyOS, an evaluation that we have not found in related work.

The rest of this paper is organized as follows. Section 2 briefly recalls FTSP as needed for this work. Section 3 describes FTSP+. Section 4 brings our experiment results, and finally, Section 5 brings the concluding remarks.

2. Flooding Time Synchronization Protocol

This section briefly describes the Flooding Time Synchronization Protocol (FTSP). For more detailed information, refer to [Maróti et al. 2004].

FTSP is a synchronization protocol for WSN that provides high accuracy, consumes few resources, uses little bandwidth and is fault-tolerant. It elects a node as root to provide the time reference for synchronization; if root failure is detected (using timeouts), another root is elected. Root and synchronized nodes send synchronization messages periodically, and receiving nodes use these messages to synchronize. Therefore, FTSP supports multi-hop networks.

Synchronization messages comprise a *sender timestamp* which is the estimated global time and *rootID* which is the network identifier of the root (where the node with the lowest ID is the chosen root). *seqNum* is a sequence counter that is incremented each synchronization round; this field is used to verify the redundancy of messages [Maróti et al. 2004].

All nodes think they are root when the network starts, so they broadcast synchronization messages to the network. When they receive a synchronization message, they check who has the lowest ID: if the local ID is higher, this node gives up on being root and starts synchronizing. Another important check is the *seqNum*. If it is greater than the local value *highestSeqNum*, it means that this is a new synchronization message and starts the synchronization procedure.

The synchronization procedure consists of computing a linear regression [Elson and Estrin 2003] that will provide the clock skew (used to estimate the global time) in relation to the reference node. The last step is to forward its local (synchronized) time to other nodes.

```

1 event Radio.receive(TimeSyncMsg *msg){
2     if( msg->rootID < myRootID )
3         myRootID = msg->rootID;
4     else if( msg->rootID > myRootID
5         || msg->seqNum <= highestSeqNum )
6         return;
7     highestSeqNum = msg->seqNum;
8     if( myRootID < myID )
9         heartBeats = 0;
10    if( numEntries >= NUMENTRIES_LIMIT
11        && getError(msg) > TIME_ERROR_LIMIT )
12        clearRegressionTable();
13    else
14        addEntryAndEstimateDrift(msg);
15 }

```

Figure 1. FTSP Receive Routine [Maróti et al. 2004]

Figure 1 shows the routine of receiving synchronization messages. Lines 2 and 3 compare the *rootID* of the synchronization message with the local *rootID*. If the message has a lower *rootID*, the node assumes this *rootID* as root. Lines 4 to 7 ignore messages with higher *rootID* and lower *seqNum*. If *seqNum* is higher, local *highestSeqNum* is updated. Lines 8 and 9 makes a root node give up on being root when it has a *rootID* lower than its own ID. In case the *rootID* is larger it is checked whether the *seqNum* is greater or equal to the value of *highestSeqNum*, this check prevents information redundancy because the message will only be used when the *rootID* is less than or equal to *myRootID* and the number greater than the value of *highestSeqNum*. Lines 10 to 15 verify if the time of a message is in disagreement with the earlier estimates of global time, if applicable clear the regression table, if not, accumulate synchronization messages to calculate the linear regression and synchronize.

```

1 event Timer.fired() {
2     ++heartBeats;
3     if( myRootID != myID
4         && heartBeats >= ROOT_TIMEOUT )
5         myRootID = myID;
6     if( numEntries >= NUMENTRIES_LIMIT
7         || myRootID == myID ) {
8         msg.rootID = myRootID;
9         msg.seqNum = highestSeqNum;
10        Radio.send(msg);
11    if( myRootID == myID )
12        ++highestSeqNum;
13    }
14 }

```

Figure 2. FTSP Send Routine [Maróti et al. 2004]

Figure 2 shows the sending routine. A node decides to be root because has not received a synchronization message for `ROOT_TIMEOUT` (lines 3 to 5). A node sends synchronization messages if it is root or has synchronized (lines 6 to 10). If a node is root, it also has to increment its *highestSeqNum*.

3. FTSP+

In any distributed time synchronization technique, nodes have to tell each other their local time. Figure 3 depicts this scenario. The sending node stores its local time $t1'$ in the synchronization message at time $t1$ and orders the message to be sent. Due to medium random access uncertainties, the sending node only gets access to the medium at time $t2$ and starts sending the message. $t2 - t1$ is called *medium access time*. The message propagates over the medium for an interval of time tp until it reaches the receiver radio at time $t3$. tp is the *propagation time*. Due to interrupt handling policies and packet header processing, the receiver node timestamps the message receipt at time $t4$ with its local time $t4''$. $t4 - t3$ is the *processing time*.

Synchronization inaccuracy happens because the receiving node thinks that at time $t4$ the sender has time $t1'$ and the receiver has time $t4''$. We can see in Figure 3 that this is not true. At time $t4$, the sending node has time $t4' = t1' + \text{medium access time} + \text{propagation time} + \text{processing time}$. MAC layer time-stamping makes *medium access time* and *processing time* equal zero. Since *propagation time* is negligible ($\sim 1\mu s$) [Maróti et al. 2004], some synchronization policies — including FTSP — can achieve very good accuracy. However, without MAC layer time-stamping, these times are non-negligible and have to be calculated.

FTSP+ calculates *medium access time* using an interrupt handler to time stamp the moment that the node gets medium access to send the synchronization message. Although medium access is granted at time $t2'$, *medium access timestamp* equals $t2' + \delta$, where δ is the overhead to process the interrupt handler.

The sender sends a correction message with content $t2' + \delta - t1'$ so the receiver

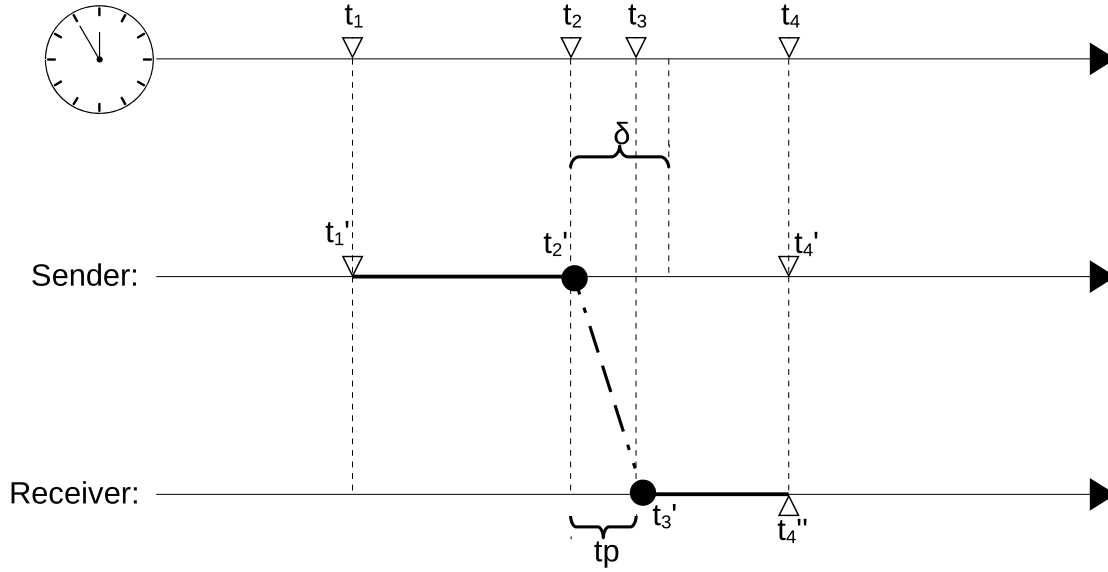


Figure 3. Synchronization steps.

can estimate $t4'$. Let us call this estimation $\bar{t4}'$. The receiver calculates $\bar{t4}'$ as in Equation (1).

$$\begin{aligned} \bar{t4}' &= t1' - (t2' + \delta - t1') \\ \bar{t4}' &= t1' + \text{medium access time} + \delta \end{aligned} \quad (1)$$

The difference between $t4'$ and $\bar{t4}'$, which is the estimation error, is:

$$t4' - \bar{t4}' = \text{propagation time} + \text{processing time} - \delta \quad (2)$$

Since *processing time* and δ are interrupt handler processing latencies, they tend to cancel out each other. We investigate their values in our experiments in Section 4. Remember that *propagation time* is negligible.

As can be seen in Figure 4, an FTSP+ receiver has routines to handle two different types of income messages: `Radio.receiveSyncMsg(SyncMsg *msg)` handles synchronization messages and `Radio.receiveCorrectionMsg(CorrectionMsg *msg)` handles correction messages. The *correction time*, expressed in Equation (3), is the information that is transmitted to the receiver to apply the correction to previous messages.

$$\text{correction time} = t2' - t1' + \delta \quad (3)$$

The `Radio.receiveSyncMsg(SyncMsg *msg)` differs from FTSP receive only for lines 13 and 14. These lines store synchronization messages in a list to wait for the correction messages if MAC layer time-stamping is not available.

```
1 event Radio.receiveSyncMsg(SyncMsg *msg) {
2     if( msg->rootID < myRootID )
3         myRootID = msg->rootID;
4     else if( msg->rootID > myRootID
5         || msg->seqNum <= highestSeqNum )
6         return;
7     highestSeqNum = msg->seqNum;
8     if( myRootID < myID )
9         heartBeats = 0;
10    if( numEntries >= NUMENTRIES_LIMIT
11        && getError(msg) > TIME_ERROR_LIMIT )
12        clearRegressionTable();
13    else if (MAC_Time==false){
14        addToWaitCorrectionMsgList(msg);
15    }else{
16        addEntryAndEstimateDrift(msg);
17    }
18 }
19
20 event Radio.receiveCorrectionMsg(CorrectionMsg *msg) {
21     applyCorrection(msg);
22     addEntryAndEstimateDrift(msg);
23 }
```

Figure 4. Receiver algorithm

The event `Radio.receiveCorrectionMsg(CorrectionMsg *msg)` receives the message with the correction value, use the function `applyCorrection(msg)` that applies the correction and removes from the wait list the synchronization message that matches the incoming correction message, corrects the timestamp and calls function `addEntryAndEstimateDrift(msg)`.

```

1 event Timer.fired() {
2     ++heartBeats;
3     if( myRootID != myID
4         && heartBeats >= ROOT_TIMEOUT )
5         myRootID = myID;
6     if( numEntries >= NUMENTRIES_LIMIT
7         || myRootID == myID ){
8         msg.rootID = myRootID;
9         msg.seqNum = highestSeqNum;
10        initialTime = call getLocalTime();
11        msg.timestamp = initialTime;
12        Radio.send(msg);
13        if( myRootID == myID )
14            ++highestSeqNum;
15    }
16 }
17
18 event sendDone(SyncMsg *msg, error_t error){
19     finalTime = call getLocalTime();
20     correctionMsg.correction = finalTime-initialTime;
21     correctionMsg.seqNum = msg.seqNum;
22     Radio.send(correctionMsg);
23 }

```

Figure 5. Sender algorithm

As can be seen in Figure 5, an FTSP+ sender has two routines: `Timer.fired()` periodically sends synchronization messages (like in FTSP) and `sendDone(message_t *msg, error_t error)` sends the correction message.

The function `Timer.fired()` differs from FTSP only for lines 10 and 11, which collects the timestamp at application level. `sendDone(message_t *msg, error_t error)` handles the interrupt when wireless medium access is granted to collect the medium access timestamp by compute the time that has passed between `send/sendDone` and send out the correction message, this technique is previously discussed by [Sousa et al. 2014] that is a simple technique for local delay estimation in WSN.

Lines 20-21 show how the correction time is computed. `seqNum` is used to identify which message is being adjusted, `finalTime` and `initialTime` refers to times t_2 and t_1 in Equation (3).

4. Evaluation

In this section, we describe our experiments to assess FTSP+'s accuracy. We have implemented FTSP+ on TinyOS 2.1.2 [Levis et al. 2005] and ran our tests on Micaz motes [MEMSIC 2015]. Micaz motes do support MAC layer timestamping, but we need this information to measure our correction accuracy. For synchronization, we overwrite MAC timestamp with our application layer timestamp.

We use *jiffys* as time unit because this is the time basis of TinyOS and represents the time between 2 clock ticks. Our experiments results correspond to 10 minutes experiments with synchronization messages being sent every 3 seconds, with 2 nodes communicating and a base station connected to an computer via serial port for record the messages.

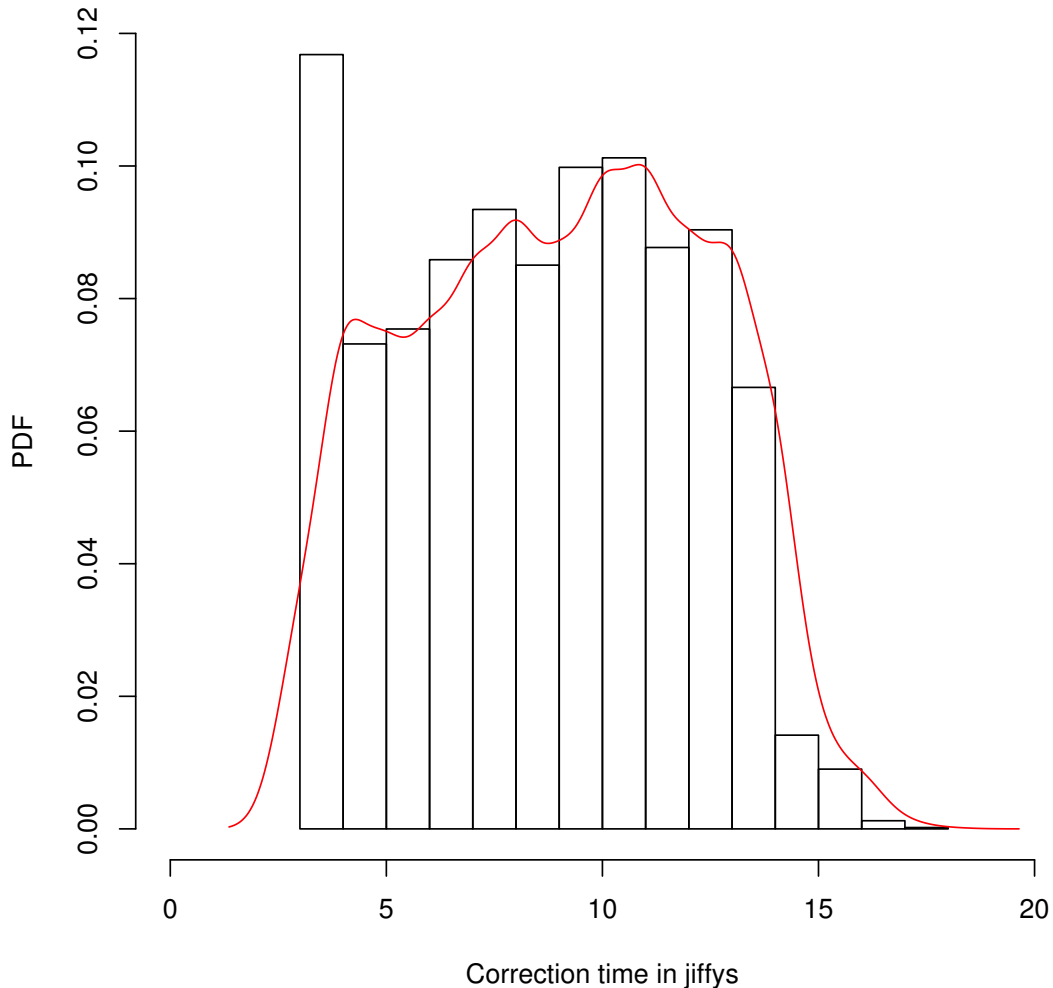


Figure 6. Histogram and P.D.F. of correction times.

Our first experiment measures the probability distribution of correction times (recall from Section 3 that it is $t_2 - t_1 + \delta$). As can be seen in Figure 6, correction time

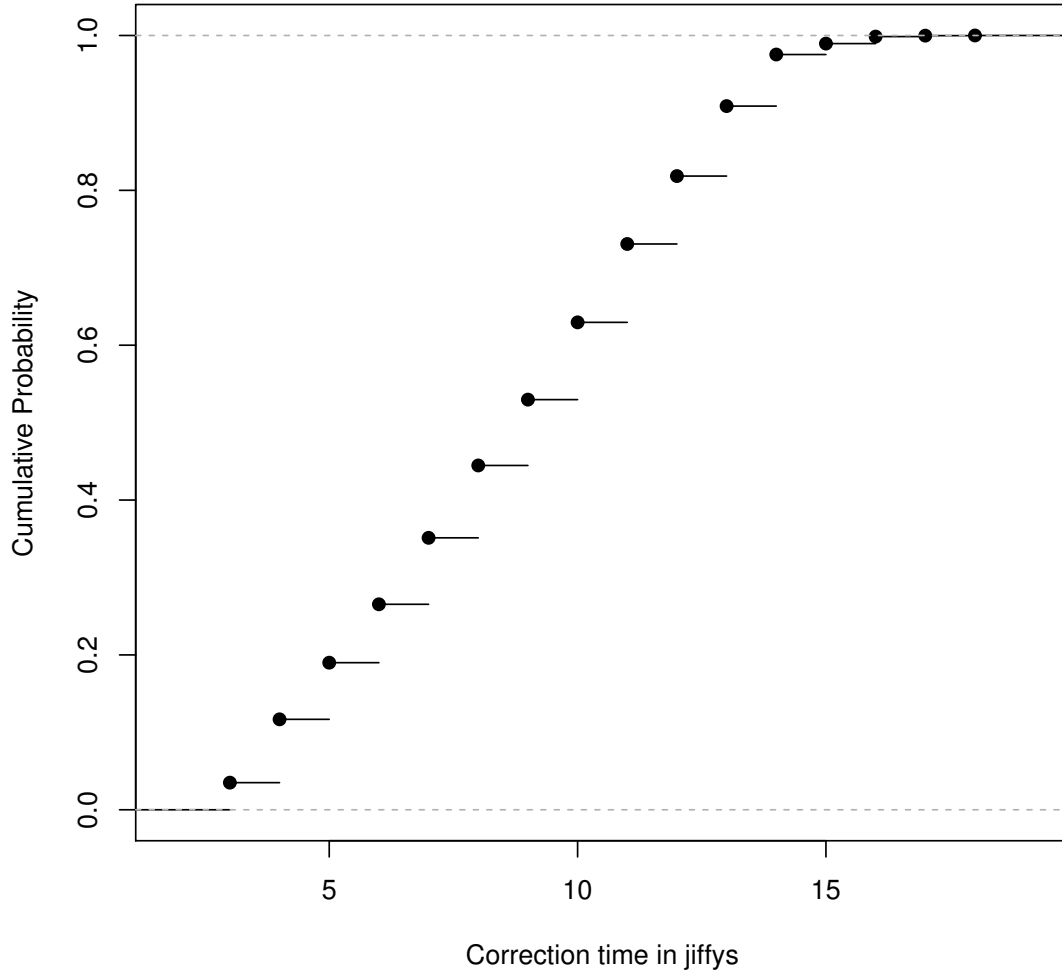


Figure 7. C.D.F. of correction times.

	mean (μs)	std. deviation
processing time	0.87 ± 0.0095	0.33
δ	0.88 ± 0.0093	0.33
correction time	9.01 ± 0.093	3.32
medium access	8.13 ± 0.093	3.31
$t4' - \bar{t4}'$	-0.0047 ± 0.013	0.47

Table 1. Mean and standard deviation

varies mostly from 5 to 13 jiffys with an uniform distribution. We can see in Figure 7, which plots the cumulative density function of the correction times, that in about 80% of cases correction time is above 5 jiffys. This is a significant overhead and source of inaccuracy that FTSP+ is able to measure and compensate for. For our scenario the average

correction time delay is 9.01^1 jiffys as we can see in Table 1 where we have the mean with confidence interval of 95% and the standard deviation.

Delay in jiffys	0	1	2	3	4	5
Frequency	606	4271	0	1	1	1

Table 2. Delay frequency of receiver *processing time*.

Delay in jiffys	0	1	2	3	4	5
Frequency	581	4297	0	0	1	1

Table 3. Delay frequency for δ .

Our second experiment measures the probability distribution of δ and *processing time*. Recall from Section 3 that they are the radio interrupt processing time in the sender and the receiver, respectively. We can see in Table 2 and 3 that their values mostly range between 0 and 1 jiffy, with extremely rare cases that do not go beyond 5 jiffys in our experiments. Their averages are 0.87 and 0.88 jiffys, which can be considered equal. Therefore, those delays, that FSTP+ is not able to calculate and compensate for, are very small and do not compromise synchronization accuracy significantly.

We calculate an estimation of the error and compare our results with other protocols (some use MAC timestamp, some do not) and summarize the results in Table 4. Knowing MAC timestamping has an inherent 1 jiffy variation in accuracy and that FTSP has $1.5\mu s$ error per hop, we use a simple rule of three to estimate FTSP+ synchronization error per hop. FTSP and FSTP+ differ only for their timestamping technique, and as can be seen in Table 1 FTSP+ timestamping is on average 0.0047 jiffy higher than MAC timestamping ($t4' - \bar{t4'}$). Therefore, our rule of three is given in Equation 4.

$$\frac{1.5\mu s}{\text{FTSP+ sync error}} = \frac{1\text{jiffys}}{(0.0047 + 1)\text{jiffys}} \quad (4)$$

We also estimate what would be the error of FTSP without MAC timestamping (we call it mFTSP in the table), in which case timestamping is on average 9 jiffys higher than MAC timestamping (*medium access delay* + *processing delay* = $8.13 + 0.87 = 9$). Equation 5 shows the rule of three for this estimation. As can be seen, RBS has a synchronization error about 10 times bigger than FTSP+.

$$\frac{1.5\mu s}{\text{mFTSP+ sync error}} = \frac{1\text{jiffys}}{(9 + 1)\text{jiffys}} \quad (5)$$

In TinyOS 1.x, where FTSP was first implemented and tested [Maróti et al. 2004], jiffys are in microsecond resolution by default. A jiffy in TinyOS 2.1.2 is approximately $1ms$, but there are compiler *TMilli* options to allow microsecond resolution. In future

¹This value is scenario-dependent while it depends upon the scenario contention level.

¹This value is scenario-dependent while it depends upon the scenario contention level.

²Values estimated based on average error in jiffys.

³Based in [Ranganathan and Nygard 2010].

MAC Timestamping		App Timestamping		
FTSP	PulseSync	FTSP+	mFTSP	RBS
$1.5\mu s^3$	$1.5\mu s^3$	$1.508\mu s^2$	$15\mu s^{1\ 2}$	$29\mu s^3$

Table 4. Average synchronization error per hop.

work, we want to use jiffies with microsecond resolution and measure the synchronization error per hop using a setup with GPIO pins for tracking events in an external and stable accuracy clock that record every exchanged messages.

4.1. Energy Consumption

The energy consumption was analyzed in experiments using the resources available on the Testbed [Lim et al. 2015]. Running the previous experiments we find the following result listed in Table 5.

FTSP	$99.04\ mJ$
FTSP+	$99.80\ mJ$

Table 5. Mean of consumption

The values represent the power consumption of CPU and radio communication of network (include the root and client node). We find close values in the tests, they are also related to synchronization frequency used on the network, for each synchronization message of FTSP the FTSP+ send another for correction, causing an extra consumption.

5. Conclusion

In this work, we presented a modified version of Flooding Time Synchronization Protocol to work without MAC layer timestamping. To keep accuracy as high as possible, we use radio interrupts to measure the instant nodes get medium access. Senders use correction messages in addition to synchronization messages in order to compensate their synchronization timestamps with medium access delay.

In our experiments, we ran tests on Micaz motes running TinyOS 2.1.2 to measure correction times, processing times and medium access time. We showed that medium access time is the main source of synchronization error and quantified it in a real testbed. We also showed that processing times are very small, on average 0.87 jiffies.

In future work, we want to precisely measure the synchronization error using a high accuracy external clock to measure timing error between synchronization events among the network motes. We also intend to consolidate the power consumption tests for different scenarios and see how much the variation in synchronization parameters affect in energy expenditure of WSN.

References

- [Arampatzis et al. 2005] Arampatzis, T., Lygeros, J., and Manesis, S. (2005). A survey of applications of wireless sensors and wireless sensor networks. In *IEEE International Symposium on Intelligent Control, Mediterrean Conference on Control and Automation*, pages 719–724. IEEE.
- [Baronti et al. 2007] Baronti, P., Pillai, P., Chook, V. W. C., Chessa, S., Gotta, A., and Hu, Y. F. (2007). Wireless sensor networks: A survey on the state of the art and the 802.15.4 and zigbee standards. *Comput. Commun.*, pages 1655–1695.
- [Cardell-Oliver et al. 2004] Cardell-Oliver, R., Smettem, K., Kranz, M., and Mayer, K. (2004). Field testing a wireless sensor network for reactive environmental monitoring [soil moisture measurement]. In *Intelligent Sensors, Sensor Networks and Information Processing Conference, 2004. Proceedings of the 2004*, pages 7–12.
- [Chong et al. 2014] Chong, F., Heck, M., Ranganathan, P., Saleh, A., and Wassel, H. (2014). Data center energy efficiency:improving energy efficiency in data centers beyond technology scaling. *Design Test, IEEE*, 31(1):93–104.
- [Elson et al. 2002] Elson, J., Girod, L., and Estrin, D. (2002). Fine-grained network time synchronization using reference broadcasts. *SIGOPS Oper. Syst. Rev.*, 36(SI):147–163.
- [Elson and Estrin 2003] Elson, J. E. and Estrin, D. (2003). *Time synchronization in wireless sensor networks*. PhD thesis, University of California, Los Angeles.
- [Ganeriwal et al. 2003] Ganeriwal, S., Kumar, R., and Srivastava, M. B. (2003). Timing-sync protocol for sensor networks. In *1st International Conference on Embedded Networked Sensor Systems*, pages 138–149. ACM.
- [Gui and Mohapatra 2004] Gui, C. and Mohapatra, P. (2004). Power conservation and quality of surveillance in target tracking sensor networks. In *Proceedings of the 10th Annual International Conference on Mobile Computing and Networking, MobiCom '04*, pages 129–143. ACM.
- [Lenzen et al. 2009] Lenzen, C., Sommer, P., and Wattenhofer, R. (2009). Optimal clock synchronization in networks. In *Proceedings of the 7th ACM Conference on Embedded Networked Sensor Systems*, pages 225–238. ACM.
- [Levis et al. 2005] Levis, P., Madden, S., Polastre, J., Szewczyk, R., Whitehouse, K., Woo, A., Gay, D., Hill, J., Welsh, M., Brewer, E., et al. (2005). Tinyos: An operating system for sensor networks. In *Ambient intelligence*, pages 115–148. Springer.
- [Lim et al. 2015] Lim, R., Maag, B., Dissler, B., Beutel, J., and Thiele, L. (2015). A testbed for fine-grained tracing of time sensitive behavior in wireless sensor networks. In *Local Computer Networks Workshops*.
- [Maróti et al. 2004] Maróti, M., Kusy, B., Simon, G., and Lédeczi, Á. (2004). The flooding time synchronization protocol. In *Proceedings of the 2nd international conference on Embedded networked sensor systems*, pages 39–49. ACM.
- [MEMSIC 2015] MEMSIC (2015). Micaz datasheet. <http://www.memsic.com>.
- [Nazemi Gelyan et al. 2007] Nazemi Gelyan, S., Eghbali, A., Roustapoor, L., Yahyavi Firouz Abadi, S., and Dehghan, M. (2007). Sltp: Scalable lightweight time synchronization protocol for wireless sensor network. In *Mobile Ad-Hoc and Sensor Networks*, volume 4864, pages 536–547. Springer Berlin.
- [Oliveira and Rodrigues 2011] Oliveira, L. M. and Rodrigues, J. J. (2011). Wireless sensor networks: a survey on environmental monitoring. *Journal of communications*, 6(2):143–151.

- [Ranganathan and Nygard 2010] Ranganathan, P. and Nygard, K. (2010). Time synchronization in wireless sensor networks: a survey. *International journal of UbiComp (IJU)*, 1(2):92–102.
- [Simon et al. 2004] Simon, G., Maróti, M., Lédeczi, Á., Balogh, G., Kusy, B., Nádas, A., Pap, G., Sallai, J., and Frampton, K. (2004). Sensor network-based countersniper system. In *Proceedings of the 2nd international conference on Embedded networked sensor systems*, pages 1–12. ACM.
- [Sinopoli et al. 2003] Sinopoli, B., Sharp, C., Schenato, L., Schaffert, S., and Sastry, S. S. (2003). Distributed control applications within sensor networks. *Proceedings of the IEEE*, 91(8):1235–1246.
- [Sommer and Wattenhofer 2009] Sommer, P. and Wattenhofer, R. (2009). Gradient clock synchronization in wireless sensor networks. In *International Conference on Information Processing in Sensor Networks*, pages 37–48.
- [Sousa et al. 2014] Sousa, C., Carrano, R. C., Magalhaes, L., and Albuquerque, C. V. (2014). Stele: A simple technique for local delay estimation in wsn. In *Computers and Communication (ISCC), 2014 IEEE Symposium on*, pages 1–6. IEEE.
- [van Greunen and Rabaey 2003] van Greunen, J. and Rabaey, J. (2003). Lightweight time synchronization for sensor networks. In *Proceedings of the 2Nd ACM International Conference on Wireless Sensor Networks and Applications*, pages 11–19. ACM.
- [Wang and Ahn 2009] Wang, S. and Ahn, T. (2009). Mac layer timestamping approach for emerging wireless sensor platform and communication architecture. US Patent App. 12/213,286.
- [Yang and Sikdar 2003] Yang, H. and Sikdar, B. (2003). A protocol for tracking mobile targets using sensor networks. In *Sensor Network Protocols and Applications, 2003. Proceedings of the First IEEE. 2003 IEEE International Workshop on*, pages 71–81. IEEE.
- [Zanatta et al.] Zanatta, G., Bottari, G. D., Guerra, R., and Leite, J. C. B. Building a WSN infrastructure with COTS components for the thermal monitoring of datacenters. In *Symposium on Applied Computing, SAC 2014, Gyeongju, Republic of Korea*.

Trilha Principal do SBRC 2016
Sessão Técnica 20
Segurança em Redes

Avaliação do Impacto da Segurança sobre a Fragmentação em Redes de Sensores Sem Fio na Internet das Coisas

Francisco Ferreira de Mendonça Júnior¹, Obionor de Oliveira Nóbrega², Paulo Roberto Freire Cunha¹

¹ Centro de Informática - Universidade Federal de Pernambuco (UFPE)
CEP: 50740-540 – Recife – PE – Brasil

² Departamento de Estatística e Informática – Universidade Federal Rural de Pernambuco (UFRPE)
Recife – PE – Brasil

{ffmj, prfc}@cin.ufpe.br, obionor.nobrega@ufrpe.br

Abstract: *On the Internet of Things, devices with limitations in computational and network resources are connected directly to the Internet. In some scenarios, these devices need to exchange data whose length exceeds the amount of available space on its frames, causing fragmentation, which adversely impacts the performance and lifetime of these networks. This paper presents an analysis of the impact of fragmentation in the delivery delay of messages in a unicast transmission scenario in a 6LoWPAN network. The reduction in the transmission of one fragment of each device can cause reductions of 20% in the delay. The variation of the security level is considered feasible to reduce fragmentation.*

Resumo: *Na Internet das Coisas, dispositivos com limitações de recursos computacionais e de rede estão conectados diretamente à Internet. Em alguns cenários, essas redes precisam trafegar dados cujo comprimento excede a carga útil disponível nos quadros em sua camada de enlace. Isso causa fragmentação, que impacta negativamente em seu desempenho e tempo de vida. Este trabalho apresenta uma análise do impacto da fragmentação no atraso de entrega de mensagens num cenário de transmissão unicast numa rede 6LoWPAN. A manipulação do nível de segurança é utilizada para controlar a fragmentação. A redução de um fragmento na transmissão de cada dispositivo pode causar reduções de até 20% no atraso.*

1. Introdução

A Internet das Coisas tem surgido como um dos paradigmas que serve de base para a evolução da Internet do Futuro [Gubbi et al. 2013]. Trata-se da conexão de objetos à Internet através de sensores, atuadores e tecnologias sem fio. A adoção do IPv6 permite que cada objeto seja endereçado individualmente, tornando possível que eles se comuniquem e troquem dados mesmo sem a intervenção humana.

A conexão de objetos à Internet passa pela consolidação das Redes de Sensores e Atuadores sem Fio (RSASF) e das tecnologias de conexão dessas redes com a Internet. Geralmente a conexão com a Internet é realizada através de *Gateways*, dispositivos responsáveis por coletar e armazenar dados dos sensores de uma RSASF e transmiti-los quando necessário. [Rachedi et al. 2015] [Yick et al. 2008]

Nos últimos anos surgiram iniciativas que pretendem mudar o paradigma de conectividade das RSASF para que os dispositivos conectem-se à Internet sem intermediários. A subcamada 6LoWPAN (*IPv6 over Low power Wireless Personal Area Networks*) provê conectividade com a Internet para dispositivos que tenham camada de acesso ao meio e camada física diferenciadas. A adaptação consiste em tradução de endereços, compressão de cabeçalhos e tratamento de fragmentação [Montenegro et al. 2007]. A função do *gateway* foi substituída pela presença do *6LoWPAN Border Router* (6LBR), que já não armazena dados dos sensores, mas direciona as requisições e o envio de dados diretamente para os dispositivos. Dispositivos das RSASF passam a ser chamados de *6LoWPAN nodes* (6LN). [Shelby et al. 2012]

Quando conectadas à Internet, as redes 6LoWPAN entram em contato com redes com características diferenciadas, abrindo novos campos para o estudo de seu funcionamento. Uma das diferenças está relacionada à quantidade e ao tamanho dos dados trafegados. A unidade máxima de transmissão do IPv6 é de 1280 bytes [Deering and Hinden 1998]. Quando pacotes assim precisam trafegar por redes 6LoWPAN, eles passam por um processo de fragmentação.

Redes 6LoWPAN geralmente apresentam tráfego de dados, como sinalização e controle, que não excedem o espaço disponível em seus quadros [Kushalnagar et al. 2007]. Mas existem tipos de redes que precisam enviar registros de várias medições que não cabem em um único quadro. Esses registros podem variar de centenas de bytes a kilobytes. Alguns exemplos mostrados se referem ao monitoramento de ferrovias, cujos dados podem alcançar 7 KB; algumas aplicações de monitoramento de saúde podem alcançar 512 KB; e aplicações de monitoramento de vulcões podem alcançar 256 bytes. [Ludovici et al. 2014].

A fragmentação causa impactos negativos no desempenho, no consumo energético e pode causar perda de pacotes. Reduzir a fragmentação permite economizar os recursos escassos das redes 6LoWPAN [Hummen et al. 2013], [Silva et al. 2009], [Pope and Simon 2013], [Kuryla and Schönwälder 2011], [Ludovic, 2014], [Suh et al. 2008]. Este trabalho apresenta uma análise da variação do nível de segurança e sua relação com a fragmentação. Busca-se analisar em quais situações a variação do nível de segurança reduz a fragmentação. São propostos algoritmos que identificam essas situações utilizando apenas tecnologias de comunicação e segurança já padronizadas. Além disso, são analisados cenários onde a relação entre a quantidade de dispositivos, a quantidade de fragmentos e a modalidade de envio de mensagens constituem um ambiente mais amplo que os cenários comumente encontrados na literatura.

A próxima seção apresenta algumas características do padrão 802.15.4 com foco nos mecanismos de fragmentação e segurança. Na seção 3 serão discutidos trabalhos que tratam sobre fragmentação, suas falhas e as relações com o presente trabalho. Na seção 4 é feita uma análise da relação entre fragmentação e segurança. Busca-se investigar situações onde a redução do nível de segurança reduz a fragmentação, principalmente através da apresentação de equações que ajudam a identificar essas situações. Nas seções 5 e 6 se discute, através de um experimento e da apresentação dos resultados, a análise do impacto de fragmentação quando causada pela aplicação de segurança. Na seção 7 são apresentadas considerações finais e propostas para trabalhos futuros.

2. Padrão 802.15.4 – Fragmentação e Segurança

O padrão IEEE 802.15.4 especifica as camadas MAC e física para redes pessoais de baixa taxa de transferência (*Low Rate Wireless Personal Area Networks*, LR-WPAN) de. O foco da especificação foi manter simplicidade de implementação, baixo custo e baixo consumo energético [Daidone et al 2014]. Essas características permitiam uma ampla adoção do padrão pela comunidade de RSSF. [Sastry et al. 2004]

O padrão prevê 3 tipos de camada física que funcionam nas frequências 868 MHz (mega-hertz) , 915 MHz ou 2.4 GHz (giga-hertz). É possível alcançar velocidades de 20 kbps (kilobits por segundo), 40 kbps e 250 kbps, respectivamente, em cada frequência de operação. Independente da frequência de operação, a camada MAC utiliza o mesmo formato de quadro. Esse quadro possui, no máximo, 127 bytes [Callaway et al 2002]. As características diferenciadas de largura de banda e de formato de quadros fazem com que o padrão 802.15.4 necessite da subcamada 6loWPAN para se conectar à Internet.

2.1. Fragmentação em Redes 802.15.4

Quando um dado vindo da subcamada 6loWPAN não couber em apenas um quadro 802.15.4, ele deve ser fragmentado. Para isso é adicionado um cabeçalho auxiliar de fragmentação. Qualquer mecanismo de segurança, quando houver, é aplicado após o processo de fragmentação. [Raza et al. 2012][Montenegro et al. 2007]

O cabeçalho auxiliar de fragmentação varia de acordo com a quantidade de fragmentos. O primeiro fragmento recebe um cabeçalho auxiliar composto por 4 octetos. Esses octetos são divididos entre um preâmbulo de 5 bits, o campo “*datagram_size*” e o campo “*datagram_tag*”. O cabeçalho auxiliar do primeiro fragmento pode ser visto na figura 1(a). A partir do segundo fragmento, o cabeçalho auxiliar ganha o campo “*datagram_offset*”. O formato do cabeçalho auxiliar dos fragmentos subsequentes pode ser visto na figura 1(b). [Montenegro et al. 2007]

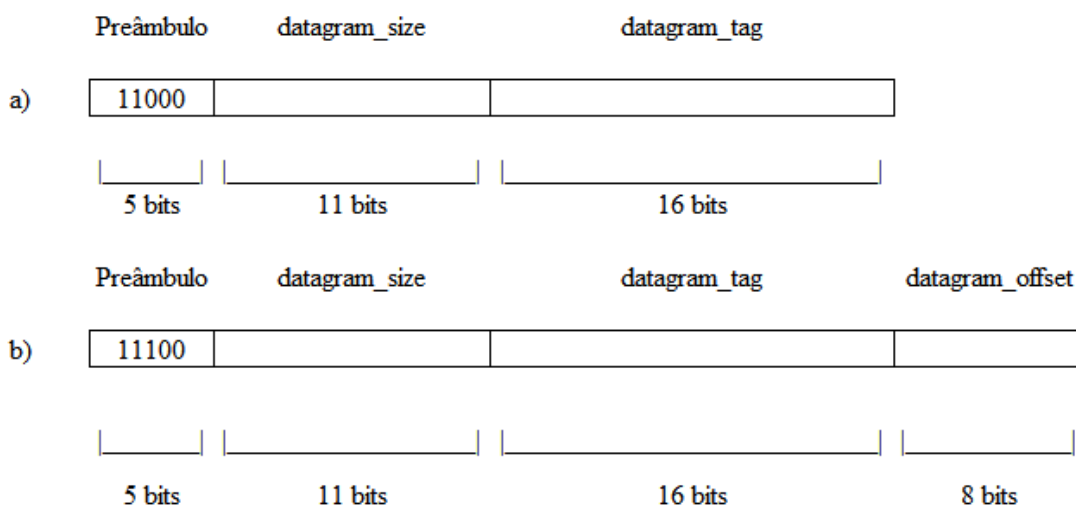


Figura 1: Formato dos cabeçalhos auxiliares de fragmentação para o primeiro fragmento (a) e para os demais fragmentos (b)

O preâmbulo identifica se aquele é o primeiro fragmento, chamado de FRAG1, ou algum fragmento subsequente, chamado de FRAGN. O campo “*datagram_size*” contém o comprimento do pacote completo, antes da fragmentação. Seu valor é comum

a todos os fragmentos, pois pode auxiliar na alocação de buffer caso os fragmentos cheguem fora de ordem. O campo “*datagram_tag*” contém um valor que identifica se um fragmento pertence a um determinado pacote. Esse valor é incrementado a cada novo pacote que necessita de fragmentação. O valor do campo “*datagram_offset*” indica o deslocamento de cada fragmento subsequente em relação ao primeiro fragmento. Ele permite que o pacote seja remontado de forma correta.

Redes 6LoWPAN são propensas a perdas e esses defeitos são agravados pela presença da fragmentação. O principal impacto para a transmissão de dados é que a fragmentação muda a modalidade de envio de mensagens. Geralmente, a taxa de geração de mensagens em uma rede é modelada por uma distribuição Poisson. Na presença de fragmentação, a rede se torna congestionada, uma vez que se considera que os fragmentos são transmitidos em rajadas [Pope and Simon 2013][Ludovic, 2014].

2.2. Segurança em Redes 802.15.4

Os mecanismos e opções de segurança no padrão 802.15.4 são conhecidos como a “subcamada de segurança” [Daidone et. al. 2014]. O padrão 802.15.4 prevê três modos de segurança: Modo Inseguro, Modo de ACL e o Modo Seguro. O Modo Inseguro não proporciona nenhuma proteção para os quadros transmitidos. No Modo de ACL (*Access Control List*), um dispositivo pode se comunicar apenas com dispositivos que estejam em uma lista pré-configurada. Quadros recebidos de outros dispositivos são descartados. Esse modo não fornece nenhuma garantia de confidencialidade, integridade ou proteção contra reenvio [Xiao et al. 2006].

O Modo Seguro prevê a utilização de 8 níveis de segurança para a camada de enlace e física. Os níveis podem ser vistos na Tabela 1. Esses níveis podem prover segurança nula, proteção de integridade, confidencialidade, ou proteção de integridade e confidencialidade combinadas. Existe também proteção contra reenvio de mensagens. [Xiao et al. 2006][Sastry et al. 2004]

Tabela 1. Níveis de segurança no padrão 802.15.4

Nível de Segurança	Descrição	Dados Criptografados	Proteção de Integridade e Autenticidade	Sobrecarga de segurança (bytes)
0	Unsecure			0
1	AES-CBC-MAC-32		X	4
2	AES-CBC-MAC-64		X	8
3	AES-CBC-MAC-128		X	16
4	AES-CTR	X		5
5	AES-CCM-32	X	X	9
6	AES-CCM-64	X	X	13
7	AES-CCM-128	X	X	21

O nível zero não prevê nenhuma proteção para as mensagens. A proteção de integridade é fornecida pela cifra AES-CBC-MAC-X. O X corresponde a extensão do Código de Integridade de Mensagem (*Message Integrity Code*, MIC). Essas mensagens não são criptografadas. Essa cifra protege a mensagem e seu cabeçalho com um código de integridade, que é calculado por blocos. O MIC é adicionado ao final da carga útil. [Sastry et al. 2004][Xiao et al. 2006]

A confidencialidade é provida pelas cifras AES-CTR e AES-CCM-X. A cifra AES-CTR protege os dados utilizando AES sobre blocos de dados. Para isso um dado é dividido em blocos de 16 Bytes. Cada bloco usa um contador diferente durante o processo de criptografia. O contador faz parte do Vetor de Inicialização (*Initialization Vector, IV*), também conhecido como *nonce*. O IV é formado pela conjunção de alguns campos como um marcador estático e o endereço do emissor. Também é formado por três contadores: o contador de quadros (4 Bytes), contador de chave (1 Byte) e o contador para os blocos (2 Bytes). O emissor inclui o contador de quadros e o contador de chave na carga útil do quadro [Sastry et al. 2004][Xiao et al. 2006]. Existem recomendações para a não utilização da cifra AES-CTR de forma isolada. A cifra não oferece proteção de integridade e de proteção contra reenvio de mensagens. [Sastry et al. 2004][Raza et al. 2012]

A cifra AES-CCM provê confidencialidade e proteção de integridade. Trata-se da aplicação dos dois mecanismos escritos anteriormente. Inicialmente é aplicado a proteção de integridade, que adiciona o MIC ao fim da carga útil. Em seguida, o processo de criptografia é aplicado sobre a carga útil e o MIC. O processo de criptografia adiciona os contadores à carga útil. Na figura 2 vemos o formato dos cabeçalhos e rodapés adicionais requeridos com a aplicação de segurança na camada física do padrão 802.15.4.

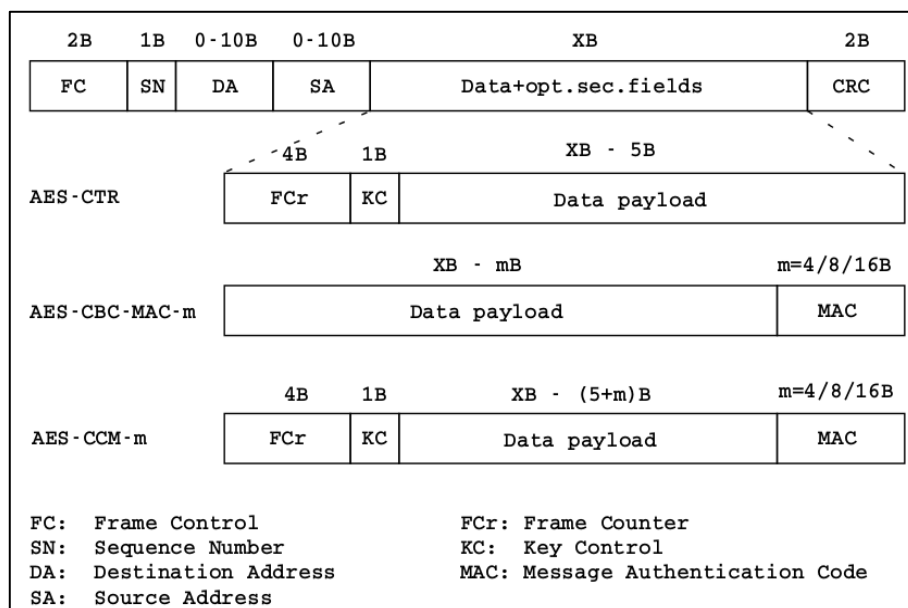


Figura 2: Formato do quadro do padrão 802.15.4 e sua relação com a segurança [Raza et al. 2012]

3. Trabalhos Relacionados

A preocupação com a fragmentação está presente desde o desenvolvimento das RSASF. Esse problema voltou à discussão quando se tornou necessário conectar esses dispositivos diretamente à Internet. Procura-se, desde então, propor mecanismos e melhorias nos mecanismos existentes para lidar com esse problema [Montenegro et al. 2007][Bormann and Shelby 2015]. Existem buscas de métodos para eliminar ou atenuar os efeitos da fragmentação, como pode ser visto em [Keoh et al. 2014]. Essa nova conectividade faz dispositivos das redes 6LoWPAN, cuja unidade máxima de

transmissão é 127 bytes, terem que lidar com o troca de dados diretamente com a Internet usando IPv6, cujo MTU (*Maximum Transmission Unity*) é de 1280 bytes [Hinden and Deering 1995].

Desde [Harvan and Schönwälder 2008] e [Cody-Kenny et al. 2008] até [Ludovici et al. 2014] essa preocupação com a fragmentação é constante. O trabalho de [Harvan and Schönwälder 2008] apresenta uma das primeiras análises do impacto da fragmentação em dispositivos limitados. Seus resultados apontam um crescimento no *round-trip time* usando ICMP (*Internet Control Message Protocol*) com mensagens *echo* que vão de 100 bytes a 1280 bytes. Apesar disso, seu cenário é simplificado com a presença de apenas um sensor. [Cody-Kenny et al. 2008] avaliaram o impacto no atraso e na perda de pacotes usando ICMP num *testbed* contendo 3 dispositivos e 1 estação base. Seus resultados mostraram aumento no atraso relacionado ao aumento no tamanho das mensagens. [Pope and Simon 2013] fizeram uma avaliação usando cenário com 16, 36 e 64 dispositivos, mas avaliaram apenas o processo de fragmentação até 2 fragmentos.

O estudo e correta avaliação do impacto na fragmentação não é necessário apenas em cenários de transmissão de dados. Alguns protocolos que auxiliem no gerenciamento, formação e manutenção de uma rede também podem precisar levá-la em consideração. [Kuryla and Schönwälder 2011] são levados a analisar o impacto da fragmentação no desenvolvimento de uma versão de SNMP para redes de dispositivos limitados. Algumas das mensagens trocadas pelo protocolo tinham tamanho próximo ou eram maiores que os 127 bytes permitidos no quadro do padrão 802.15.4. Essas mensagens sofriam fragmentação, e isso foi analisado para a avaliação do protocolo.

[Raza et al. 2013] preocupou-se com fragmentação no desenvolvimento de uma implementação de uma versão leve e segura do protocolo CoAP (*Constrained Application Protocol*). Métodos de compressão de mensagens de negociação do protocolo de segurança DTLS (*Datagram Transport Layer Protocol*) foram propostos. Mesmo assim, nem sempre era possível evitar fragmentação, pois algumas mensagens permaneciam com comprimento maior do que o máximo permitido.

O desenvolvimento de novos protocolos e a evolução dos antigos requer novas avaliações dos mecanismos de fragmentação existentes. O protocolo CoAP oferece a possibilidade de transferência de dados através de *blockwise transfer* [Bormann and Shelby 2015]. Trata-se de uma forma de dividir uma requisição ou uma resposta, de forma que cada parte seja transmitida como uma requisição independente nas camadas mais baixas da pilha de protocolos. Dessa forma, cada parte é transmitida e confirmada individualmente e pode ser retransmitida em caso de perda. Caso a requisição completa fosse enviada para as camadas inferiores e sofresse fragmentação, a perda de um dos fragmentos poderia gerar a retransmissão de toda a requisição. A retransmissão causa atrasos e aumento no consumo energético.

Pensando nisso, [Ludovic, 2014] realiza uma comparação entre fragmentação e *blockwise transfer*. Seus resultados apontam que a transmissão da requisição completa para que seja fragmentada na camada de rede pode ser mais eficiente que a utilização do mecanismo de *blockwise transfer*. Isso ocorre pois o CoAP demanda a troca de mais mensagens, gerando mais ocupação no canal que as confirmações da camada de rede.

[Rachedi et al. 2015] baseou-se no impacto da segurança para a construção de rotas pelo protocolo RPL em redes 6LoWPAN. Para isso ele se preocupou com o

impacto computacional de criptografar e descriptografar os dados. Esse impacto alimentava um controlador PID (*Proportional–Integral–Derivative controller*) para a construção de rotas do tipo AODV (*Ad-hoc On-demand Distance Vector*). Ainda assim não foram investigados os impactos que a fragmentação podia proporcionar, visto que a transmissão de dados acarreta a elevação do atraso e do consumo energético.

Apesar de estes trabalhos proporem melhorias, a maior parte dos cenários avaliados se constitui de cenários simplificados. As redes são formadas por poucos dispositivos. O modelo mais comum trata da avaliação utilizando apenas dois dispositivos, o que não condiz com a realidade de um cenário de Internet das Coisas. [Ludovic, 2014] realizou a avaliação em ambientes com mais dispositivos, mas não propôs mecanismos de eliminação ou atenuação dos efeitos da fragmentação.

4. Análise do Impacto da Fragmentação Causada pela Adição de Segurança

Visto que os cenários analisados na seção 3 são simplificados para o contexto da Internet das Coisas, é necessário avaliar a fragmentação em ambientes maiores e sob condições que simulem a troca de mensagens entre 6LN e a Internet. Técnicas que reduzam a fragmentação são necessárias, uma vez que seus impactos no aumento no atraso e taxa de entrega de mensagens são significativos [Harvan and Schönwälder 2008][Humenn, 2013][Pope and Simon 2013]. Serão analisadas as situações em que fragmentação pode acontecer devido à aplicação de segurança.

Esta análise é baseada na premissa de que a Qualidade de Serviço (*Quality of Service*, QoS) também é um fator importante para as aplicações de Internet das Coisas [Rachedi et al. 2015]. Estudos extensivos têm sido feitos em relação à ocupação de memória e consumo energético em RSASF [Yick et al. 2008]. Algumas métricas de QoS, como o atraso, tem sido deixadas em segundo plano em relação aos dois parâmetros citados.

É conhecido na literatura que o tamanho tem impactos no tempo de entrega das mensagens. Mesmo quando a fragmentação não é necessária, esse tempo pode variar dependendo da quantidade de mensagens, da quantidade de dispositivos na rede ou da quantidade de saltos. [Pope and Simon 2013][Rachedi et al. 2015]

Para esta análise serão utilizados apenas os níveis de segurança 5, 6 e 7, pois permitem o envio de dados criptografados e com proteção de integridade. Para que os dados sejam enviados com a máxima segurança habilitada é necessário acrescentar 21 bytes de informações de segurança em cada quadro, reduzindo a quantidade de dados que a aplicação pode enviar. Cada um dos níveis menores ocupa menos espaço por quadro.

Os limites da fragmentação correspondem à quantidade de bytes que podem ser economizados com a redução do nível de segurança. A quantidade de bytes necessários para a aplicação de cada nível está descrita na Tabela 1. À vista disso podem-se construir pseudocódigos para detectar os limites onde a redução do nível de segurança reduz o impacto da fragmentação.

No algoritmo 1, as variáveis “*SicsLowOneFragLen*”, “*SicsLowFragLen*” e “*SicsLowFragNLen*” representam a quantidade de carga útil que um quadro pode transportar após a adição do MIC de acordo com o nível de segurança. A variável

“*SicsLowOneFragLen*” representa a carga útil de um quadro após a adição do MIC e quando não ocorre fragmentação. Essa variável representa uma quantidade maior de carga útil, pois nenhum cabeçalho de fragmentação foi adicionado.

Algoritmo 1: Algoritmo para identificação da quantidade de fragmentos de acordo com a nível de segurança

```

Algorithm Fragmentation Threshold Level
Input   len: Data Length
1  if len <= SicsLowOneFragLen then
2    fragments = 1
3  else
4    fragments =  $\text{roundUp}((\text{len} - \text{SicsLowFrag1Len}) / \text{SicsLowFragNLen}) + 1$ 
5  return fragments

```

A variável “*SicsLowFrag1Len*” representa a carga útil do primeiro quadro de um dado fragmentado. Seu valor representa a quantidade de bytes disponíveis para dados após a adição do cabeçalho adicional de fragmentação e do MIC. Essa variável apresenta um valor maior que “*SicsLowFragNLen*”. Isso acontece porque o cabeçalho de fragmentação nos fragmentos subsequentes é maior, reduzindo o espaço para carga útil nos fragmentos adicionais.

Os valores que podem ser atribuídos as variáveis do algoritmo 1 são relacionados com “XB”, na figura 2. Esses valores são diferentes para cada nível de segurança e podem ser vistos na tabela 2. Na linha 1 do algoritmo 1, é feito o cálculo da quantidade de dados que não ativa a fragmentação. Este valor é representado pela variável “*SicsLowOneFragLen*”. Valores maiores que o especificado na coluna “*SicsLowOneFragLen*” da tabela 2 provocam a fragmentação, de acordo com o nível de segurança.

Tabela 2. Valores para as variáveis do Algoritmo 1 de acordo com o nível de segurança

	<i>SicsLowOneFragLen</i>	<i>SicsLowFrag1Len</i>	<i>SicsLowFragNLen</i>
Level 5	91	87	86
Level 6	87	83	82
Level 7	79	75	74

A presença das “*SicsLowFrag1Len*” e “*SicsLowFragNLen*” na linha 3 do Algoritmo 1 é relacionada ao cabeçalho de fragmentação, que possui valores diferentes para o primeiro fragmento e para os fragmentos seguintes. No caso do nível 5, o valor 87 para “*SicsLowFrag1Len*” indica a quantidade de dados referente ao primeiro fragmento. A divisão por 86, referente à variável “*SicsLowFragNLen*”, indica a quantidade de dados que os demais fragmentos podem transportar. Quanto maior o nível de segurança, menor é a quantidade de espaço disponível para dados. A mesma relação pode ser aplicada aos demais níveis de segurança.

O algoritmo 2 verifica se, para uma dada quantidade de dados, a redução no nível de segurança resultará na redução da quantidade de fragmentos. O acrônimo “FTL*” representa o “*Fragmentation Threshold Level **”, e faz referência ao algoritmo 1 quando este for alimentado com os valores da tabela 2, de acordo com o nível de segurança aplicado. No algoritmo 2, a quantidade de fragmentos resultante da aplicação de cada nível de segurança é comparada. Quando a quantidade de fragmentos for menor, para um nível menor de segurança, a redução no nível de segurança é acionada.

Algoritmo 2: Algoritmo para redução da quantidade de fragmentos**Algorithm Thresholds****Input** *frag7*: FTL7*frag6*: FTL6*frag5*: FTL5**1** *if frag5 < frag6 || frag6 < frag7 || frag5 < frag7 then***2** *decreaseSecurityLevel()***3** *return fragments*

Na figura 3 verifica-se o impacto do tamanho da mensagem na fragmentação quando a quantidade de dados que chega a subcamada 6LoWPAN está próxima dos limites de fragmentação. O gráfico foi aprimorado para demonstrar situações onde a redução no nível de segurança reduz a quantidade de fragmentos. Isso significa que esses pontos satisfazem as condições expostas nos algoritmos 1 e 2 para a redução da fragmentação. É possível observar que existe uma tendência a uma redução cada vez maior na quantidade de fragmentos conforme o tamanho na mensagem aumente. Um mecanismo que faça utilização do nível de segurança para evitar fragmentação pode ser viável [Rachedi et al. 2015].

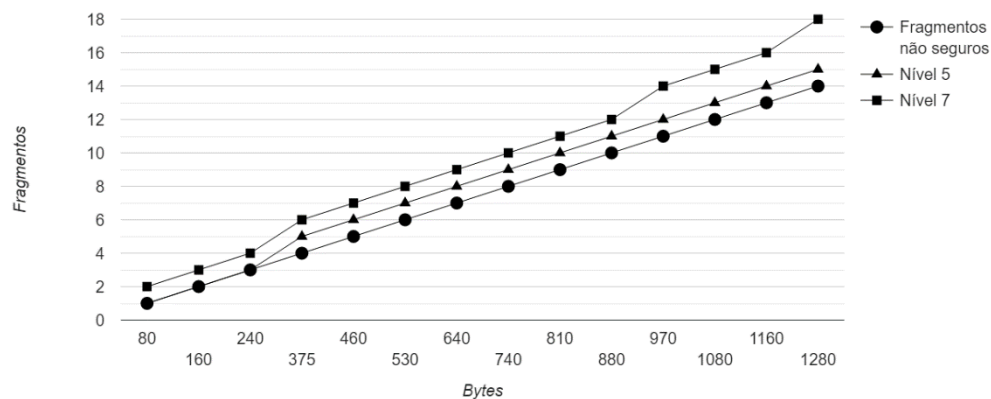


Figura 3: Aumento na quantidade de fragmentos relacionado a adição de segurança

5. Configuração do Experimento

Foram realizados experimentos para medir a economia de tempo ocasionada pela redução na quantidade de fragmentos. Os experimentos foram realizados usando ao simulador Cooja [Osterlind et al. 2006]. É possível simular dispositivos em três níveis: instruções em nível de máquina, nível de rede e sistema operacional. É possível coletar informações de cada um desses níveis.

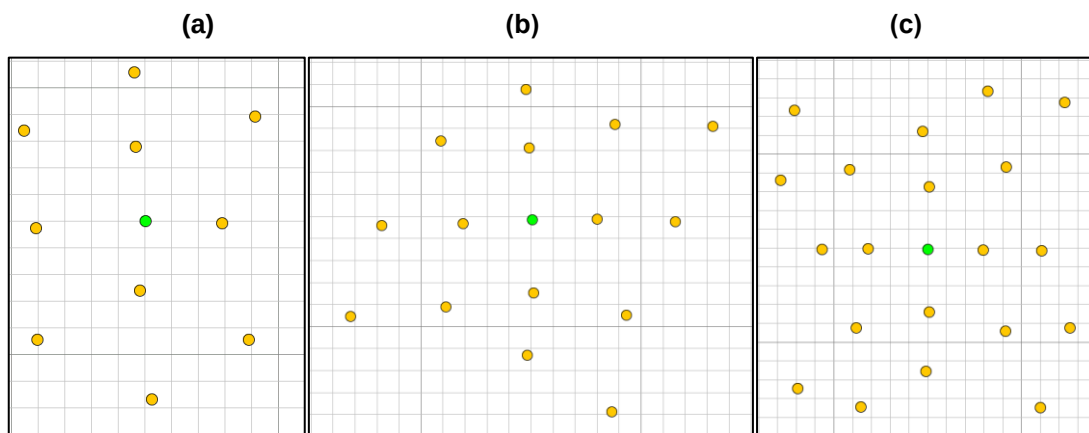
No simulador, foi configurada uma aplicação de entrega de mensagens utilizando UDP sobre 6LoWPAN. Os dados são enviados a um receptor, no centro físico da rede em estrela, que emite uma mensagem na camada de aplicação quando recebe algum dado. Os experimentos foram feitos em redes de 11, 16 e 21 dispositivos [Ludovic, 2014]. Em cada rede, um dos dispositivos faz o papel de 6LBR e fica no centro da rede. Foram investigadas três taxas de envio de mensagens: 1, 5 e 10 mpm

(mensagens por minuto) [Ludovic, 2014]. O comprimento da mensagem foi controlado para simular o comportamento da manipulação de segurança sobre dados. Foi adotada a premissa de que o tempo gasto pelas operações criptográficas é desprezível quando realizado com o auxílio de aceleradores hardware [Lee et al. 2010].

Foram realizados experimentos em modelo fatorial completo para cada rede (3), cada quantidade de fragmentos (6) e cada modalidade de envio de mensagens (3), totalizando 54 tipos para simulação. Cada uma dessas simulações foi repetida por 10 vezes. Cada experimento foi executado durante 1 hora em tempo de simulação, de forma que são geradas até 13000 mensagens. O simulador gera sementes aleatórias da ordem de 19 dígitos para cada replicação. Cada experimento foi executado durante 1 hora em tempo de simulação, de forma que são geradas até 13000 mensagens.

A partir de cada uma das 10 repetições é gerada uma média. As amostras de 10 médias provenientes de simulações de quantidades de fragmentos adjacentes são comparadas. Para a comparação, são usados testes estatísticos para a diferença de duas amostras com intervalo de confiança de 90%. A hipótese alternativa do teste é que o atraso medido na rede que transmite mais fragmentos é maior que o atraso medido na rede com quantidade de fragmentos subjacente. Na figura 4 vemos os cenários da simulação para 11 dispositivos (a), 16 dispositivos (b) e 21 dispositivos (c).

Figura 4: Cenário da simulação



6. Análise dos Resultados

Pode-se observar, nos gráficos das figuras 5, 6 e 7, redução da fragmentação apresenta redução no atraso. Esses valores representam a redução no atraso médio quando o dado é enviado de um 6LN para o 6LBR. Porém, é possível observar que o aumento da quantidade de dispositivos e o aumento da quantidade de mensagens podem tornar os ganhos menos relevantes, uma vez que a rede está congestionada o tempo todo.

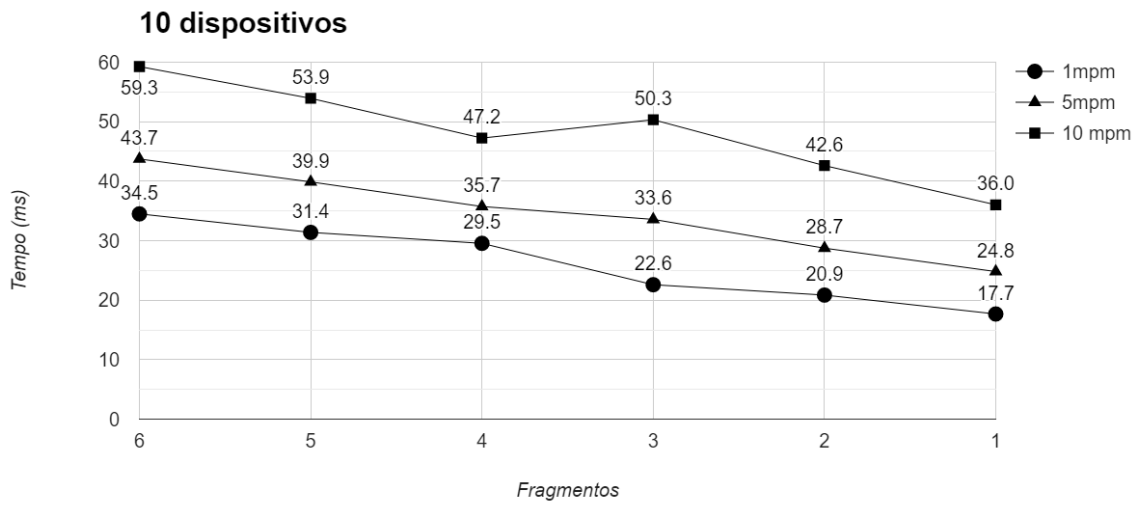


Figura 5: Redução média no atraso em redes de 10 dispositivos sob diferentes taxas de geração de dados

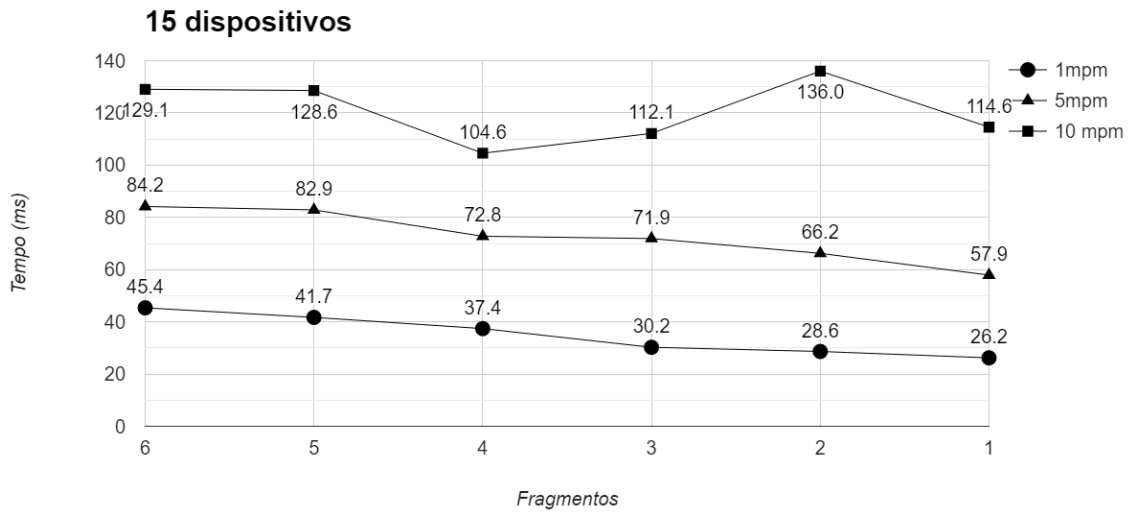


Figura 6: Redução média no atraso em redes de 15 dispositivos sob diferentes taxas de geração de dados

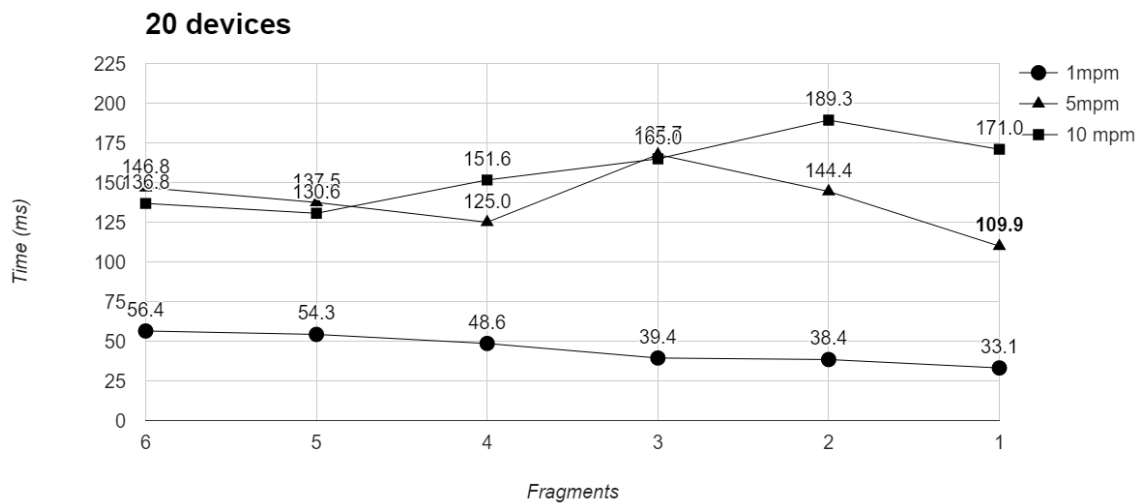


Figura 7: Redução média no atraso em redes de 10 dispositivos sob diferentes taxas de geração de dados

Na figura 5 é possível observar a redução no atraso médio da rede quando ocorre a redução de segurança na rede de 10 dispositivos. Nas modalidades de 1 mpm e 5 mpm todas as reduções apresentam ganhos significativos. Nas figuras 5, 6 e 7, no envio de 10 mpm, não existem ganhos estatisticamente significativos, apesar de existirem ganhos percentuais. Esse comportamento ocorre devido ao congestionamento da rede [Pope and Simon 2013] [Ludovic, 2014].

Redes 6LoWPAN cuja frequência de geração de dados está entre 1 e 5 mpm apresentam decréscimo significativo no atraso quando ocorre redução da fragmentação. Redes com até 10 dispositivos apresentam ganhos em todas as taxas.

Em redes com 15 e 20 dispositivos, os ganhos são expressivos quando o tráfego da rede é baixo. Nas redes com 15 dispositivos, os ganhos existem quando as mensagens são enviadas a, no máximo, 5 mpm. Até esse limite, são encontrados ganhos próximos a 20%. Nas redes de 20 dispositivos, os ganhos se concentram quando as mensagens são enviadas a 1 mpm. Quando a quantidade de mensagens aumenta, os ganhos passam a ser menos perceptíveis. Mesmo assim ainda são encontrados ganhos próximos a 20% nessas redes.

7. Considerações Finais e Trabalhos Futuros

Este trabalho avaliou as situações em que a redução do nível de segurança do padrão 802.15.4 auxilia na redução dos efeitos da fragmentação no atraso de redes 6LoWPAN. Foram propostos algoritmos para identificar essas situações e acionar a redução no nível de segurança. Foi possível analisar que existem casos em que ocorrem reduções de até 20% no atraso. As reduções ocorrem com mais significância em redes de 10 dispositivos. Em redes de 15 e 20 dispositivos os ganhos são significativos quando a taxa de transmissão de mensagens não ultrapassa 5 mpm. Conclui-se que a redução do nível de segurança é método viável para reduzir fragmentação e reduzir o atraso em redes 6LoWPAN.

Trabalhos futuros podem propor outras técnicas que utilizem a flexibilidade existente na subcamada de segurança do padrão 802.15.4, como demonstrado na seção 2.2, para economizar energia e diminuir o atraso no envio das mensagens. Podem ser

investigadas mais situações onde ganhos são expressivos. Também podem ser estudados os casos em que a criptografia é realizada por software, uma vez que parte dos rádios ainda não fornece suporte completo à criptografia realizada por hardware.

Referências

- Bormann, C., & Shelby, Z. (2015). Blockwise Transfers in CoAP draft-ietf-core-block-18. Available online: <http://tools.ietf.org/html/draft-ietf-core-block-18> (accessed on 25 nov 2015).
- Cody-Kenny, B., Guerin, D., Ennis, D., Simon Carbajo, R., Huggard, M., & Mc Goldrick, C. (2009, October). Performance evaluation of the 6LoWPAN protocol on MICAz and TelosB motes. In *Proceedings of the 4th ACM workshop on Performance monitoring and measurement of heterogeneous wireless and wired networks* (pp. 25-30). ACM.
- Callaway, E., Gorday, P., Hester, L., Gutierrez, J. A., Naeve, M., Heile, B., & Bahl, V. (2002). Home networking with IEEE 802. 15. 4: a developing standard for low-rate wireless personal area networks. *IEEE Communications magazine*, 40(8), 70-77.
- Daidone, R., Dini, G., Anastasi, G. (2014). On evaluating the performance impact of the IEEE 802.15. 4 security sub-layer. *Computer Communications*, 47, 65-76.
- Deering, S. E. Hinden, R (1998). Internet protocol, version 6 (IPv6) specification. (No. RFC 2460).
- Gubbi, J., Buyya, R., Marusic, S., & Palaniswami, M. (2013). Internet of Things (IoT): A vision, architectural elements, and future directions. *Future Generation Computer Systems*, 29(7), 1645-1660.
- Harvan, M., & Schönwälder, J. (2008). TinyOS Motes on the Internet: IPv6 over 802.15. 4 (6LoWPAN). *PIK-Praxis der Informationsverarbeitung und Kommunikation*, 31(4), 244-251.
- Hinden, R., & Deering, S. (1995). Internet protocol, version 6 (IPv6) specification.
- Hummen, R., Hiller, J., Wirtz, H., Henze, M., Shafagh, H., & Wehrle, K. (2013, April). "6LoWPAN fragmentation attacks and mitigation mechanisms". In *Proceedings of the sixth ACM conference on Security and privacy in wireless and mobile networks* (pp. 55-66). ACM.
- Keoh, S. L., Kumar, S. S., & Tschofenig, H. (2014). Securing the internet of things: A standardization perspective. *Internet of Things Journal, IEEE*, 1(3), 265-275.
- Kuryla, S., & Schönwälder, J. (2011). Evaluation of the resource requirements of SNMP agents on constrained devices. In *Managing the Dynamics of Networks and Services* (pp. 100-111). Springer Berlin Heidelberg.
- Kushalnagar, N., Montenegro, G., & Schumacher, C. (2007). IPv6 over low-power wireless personal area networks (6LoWPANs): overview, assumptions, problem statement, and goals (No. RFC 4919).
- Lee, J., Kapitanova, K., & Son, S. H. (2010). The price of security in wireless sensor networks. *Computer Networks*, 54(17), 2967-2978.
- Ludovici, A., Marco, P. D., Calveras, A., & Johansson, K. H. (2014). Analytical model

- of large data transactions in CoAP networks. *Sensors*, 14(8), 15610-15638.
- Montenegro, G., Kushalnagar, N., Hui, J., & Culler, D. (2007). Transmission of IPv6 packets over IEEE 802.15. 4 networks (No. RFC 4944).
- Osterlind, F., Dunkels, A., Eriksson, J., Finne, N., & Voigt, T. (2006, November). Cross-level sensor network simulation with Cooja. In *Local Computer Networks, Proceedings 2006 31st IEEE Conference on* (pp. 641-648). IEEE.
- Pope, J., & Simon, R. (2013). The Impact of Packet Fragmentation and Reassembly in Resource Constrained Wireless Networks. *CIT. Journal of Computing and Information Technology*, 21(2), 97-107.
- Rachedi, A., & Hasnaoui, A. (2015). Advanced quality of services with security integration in wireless sensor networks. *Wireless Communications and Mobile Computing*, 15(6), 1106-1116.
- Raza, S., Duquennoy, S., Höglund, J., Roedig, U., & Voigt, T. (2012). Secure communication for the Internet of Things—a comparison of link layer security and IPsec for 6LoWPAN. *Security and Communication Networks*.
- Raza, S., Shafagh, H., Hewage, K., Hummen, R., & Voigt, T. (2013). Lithe: Lightweight secure CoAP for the internet of things. *Sensors Journal, IEEE*, 13(10), 3711-3720.
- Sastry, N., & Wagner, D. (2004, October). Security considerations for IEEE 802.15. 4 networks. In *Proceedings of the 3rd ACM workshop on Wireless security* (pp. 32-42). ACM.
- Shelby, Z., Chakrabarti, S., Nordmark, E., & Bormann, C. (2012). Neighbor discovery optimization for IPv6 over low-power wireless personal area networks (6LoWPANs) (No. RFC 6775).
- Silva, R., Silva, J. S., & Boavida, F. (2009). Evaluating 6LoWPAN implementations in WSNs. *Proceedings of 9th Conferencia sobre Redes de Computadores Oeiras, Portugal*, 21.
- Suh, C., Mir, Z. H., & Ko, Y. B. (2008). Design and implementation of enhanced IEEE 802.15. 4 for supporting multimedia service in Wireless Sensor Networks. *Computer Networks*, 52(13), 2568-2581.
- Xiao, Y., Chen, H. H., Sun, B., Wang, R., & Sethi, S. (2006). MAC security and security overhead analysis in the IEEE 802.15. 4 wireless sensor networks. *EURASIP Journal on Wireless Communications and Networking*, 2006(2), 81-81.
- Yick, J., Mukherjee, B., & Ghosal, D. (2008). Wireless sensor network survey. *Computer networks*, 52(12), 2292-2330.